

DiffEngine Dashboard: A MERN–C++ Hybrid Platform for Multi-Algorithm Code Differencing and Structural Analysis

Richeeck Ghosh, Utkarsh Raj, Priyanshu Paliwal

Indian Institute of Technology Kharagpur

Abstract—This paper describes the design, architecture, and implementation of *DiffEngine Dashboard*, an open-source code-differencing and structural analysis platform that combines a compiled C++ diff engine with a MERN-stack (MongoDB, Express.js, Node.js, React) web dashboard. The system provides three distinct differencing algorithms—Myers, Histogram, and GumTree AST Diff—each implemented in C++14 and invoked through a Node.js/Express.js REST API. A dual-parser AST front-end supports both a toy C-like language and Lisp-style S-expressions, enabling structural comparison of program trees via top-down and bottom-up node matching. The React frontend renders line-based diffs in split and unified views, displays annotated AST trees side-by-side, and presents a supplementary heuristic complexity comparison panel. Comparison records are persisted in MongoDB with transparent in-memory fallback when a database connection is unavailable. The paper documents each algorithmic component, its data structures, its asymptotic complexity, and the end-to-end execution pipeline. No benchmark superiority is claimed; the system is described as implemented.

Index Terms—diff algorithms, Myers algorithm, Histogram diff, GumTree, abstract syntax tree, MERN stack, code comparison, edit distance, tree matching

I. INTRODUCTION

Comparing two versions of a text or program is a fundamental operation in software engineering. Version-control systems, code-review tools, and refactoring assistants all depend on high-quality differencing to present changes in a form that is both compact and understandable. While line-based tools such as Unix `diff` are widely used, purely textual comparison cannot capture structural relationships within source code: a renamed variable, a moved function, or a reordered block may appear as a large sequence of unrelated text changes even though the underlying structural edit is minimal.

DiffEngine Dashboard addresses this gap by integrating three complementary approaches within a single interactive workspace:

- 1) **Myers Diff** [1] — a greedy shortest-edit-path algorithm that produces a minimal sequence of line insertions and deletions in $O(ND)$ time, where N is the sum of input lengths and D is the edit distance.
- 2) **Histogram Diff** — a divide-and-conquer strategy (a patience-sort extension) that anchors the comparison on the rarest shared line in each sub-region and falls back to Myers when no shared line exists, producing more human-readable output.

- 3) **GumTree AST Diff** [2] — a two-phase tree matching algorithm that first applies top-down isomorphism detection on subtrees of equal structure, then recovers additional matches bottom-up using Dice similarity, and finally emits a four-operation edit script (INSERT, DELETE, UPDATE, MOVE) over AST nodes.

The platform also includes a supplementary heuristic complexity estimator that performs lightweight static analysis of submitted source code to estimate asymptotic time complexity, described separately in Section XI.

The system is documented here as technical reference material to accompany the open-source GitHub repository. The description is faithful to the implementation; no features or performance claims beyond those directly derivable from the source code are asserted.

A. Contributions

This work provides:

- A self-contained C++14 implementation of Myers Diff, Histogram Diff, and GumTree AST Diff, compiled to a single executable invoked via subprocess from Node.js.
- A dual-mode AST parser capable of accepting both a toy C-like language (with functions, variables, control flow, and binary expressions) and Lisp-style S-expressions, with a word-token fallback for unrecognised input.
- A JSON-over-subprocess IPC protocol between the Node.js backend and the C++ engine.
- A React dashboard with split and unified text diff views, an interactive annotated AST tree viewer, and a supplementary complexity comparison panel.
- MongoDB persistence with automatic in-memory fallback, enabling the system to run without a database.

II. RELATED WORK

A. Line-Based Differencing

The problem of computing a minimal edit sequence between two sequences was formalized by Myers [1], who presented an $O(ND)$ algorithm that explores a shortest edit path through a two-dimensional edit graph. The algorithm requires $O(N + D)$ space in its forward variant and $O(D^2)$ in the backtracking formulation used here. Git's default diff algorithm is a variant of Myers.

Bram Cohen’s patience diff and the related histogram algorithm [3] improve on Myers for human readability by preferring to anchor on unique or rare lines before recursing. Git exposes histogram diff via `-diff-algorithm=histogram`.

B. Structural / AST Differencing

Gall, Hajek, and Jazayeri [5] and later Chawathe et al. [4] established formalisms for tree-structured edit distance and edit scripts. Falleri et al. [2] introduced GumTree, which combines a top-down height-ordered subtree matching phase with a bottom-up Dice-coefficient similarity phase to produce a four-operation edit script over ASTs generated by language-specific parsers. The present system implements the GumTree matching strategy on a hand-written dual-mode parser, without integrating an external language parser.

Hashimoto and Mori [6] and Martinez et al. [7] extended AST differencing toward automated program repair and fine-grained change characterisation.

C. Interactive Diff Visualisation

Tools such as Meld, Beyond Compare, and GitHub’s file-diff viewer provide split and unified line-diff views similar to the DiffViewer component implemented here. The present dashboard combines all three differencing modes and the structural AST viewer in a single browser-based interface.

III. SYSTEM ARCHITECTURE

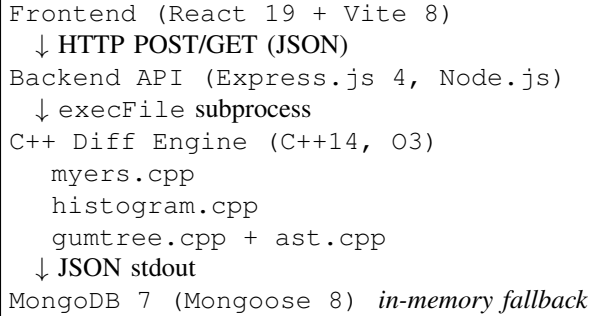


Fig. 1. High-level system architecture. Arrows denote data flow. The C++ engine is a pre-compiled executable invoked as a child process by the Node.js server.

The system is organised into three tiers (Fig. 1):

a) *C++ Diff Engine*: Seven C++ source files (`diff_core.h`, `ast.h`, `ast.cpp`, `myers.cpp`, `histogram.cpp`, `gumtree.cpp`, `main.cpp`) are compiled with `g++ -O3 -std=c++14` to produce a single executable, `diff_engine.exe`. The executable accepts three positional arguments: the algorithm name (`myers`, `histogram`, or `gumtree`), and the paths to two temporary text files. It emits a JSON document on `stdout` and exits.

b) *Node.js / Express.js Backend*: `server.js` exposes a REST API on port 5000 with three routes: `POST /api/diff`, `GET /api/history`, and `DELETE /api/history/:id`. On receiving a diff request, the server writes the two submitted text buffers to timestamped temporary files in `backend/temp/`, launches the C++ executable via

Node’s `execFile`, parses the resulting JSON, optionally persists the record in MongoDB through Mongoose, and returns the full result to the client. A heuristic complexity estimator implemented entirely in JavaScript runs on both text buffers for every diff request (and exclusively for complexity-mode requests).

c) *React Frontend*: A Vite-bundled React 19 single-page application provides the user interface on port 5173. Three specialised components handle output rendering: `DiffViewer.jsx` (line-based diffs), `ASTDiffViewer.jsx` (annotated tree view), and `ComplexityViewer.jsx` (heuristic complexity panel). The main `App.jsx` manages global state, communicates with the backend, and dispatches the appropriate viewer based on the selected algorithm.

IV. CORE DATA STRUCTURES

A. Line-Diff Record

Line-based algorithms (Myers and Histogram) produce a `std::vector<LineDiff>`, where each element is defined as:

```

1 enum class LineOpType { EQUAL, DELETE, INSERT };
2
3 struct LineDiff {
4     LineOpType type;
5     std::string line;
6     int lineNoA; // 1-based index in file A (0 if
7                 // absent)
8     int lineNoB; // 1-based index in file B (0 if
9                 // absent)
10 };

```

Listing 1. LineDiff structure from `diff_core.h`.

The serialiser in `main.cpp` encodes this vector as a JSON array and writes it to `stdout`.

B. AST Node

AST nodes are heap-allocated `shared_ptr` objects:

```

1 struct ASTNode
2 : public std::enable_shared_from_this<ASTNode> {
3     int id;
4     std::string type;
5     std::string value;
6     std::vector<std::shared_ptr<ASTNode>> children;
7     std::weak_ptr<ASTNode> parent;
8     int height;
9     int size;
10    std::string structure_hash;
11    ASTNode(int id, const std::string& type,
12            const std::string& value)
13        : id(id), type(type), value(value),
14          height(1), size(1) {}
15    void post_process();
16    std::string to_json() const;
17 };

```

Listing 2. ASTNode structure from `ast.h`.

The `post_process()` method propagates parent pointers, computes subtree sizes and heights, and builds a recursive `structure_hash` string of the form `type:value[child_hash,...]`. Two subtrees with identical `structure_hash` values are isomorphic and can be matched in the top-down phase.

C. AST Edit Script Record

```

1 enum class ASTOpType { INSERT, DELETE, UPDATE, MOVE
2 };
3 struct ASTDiffOp {
4     ASTOpType type;
5     int nodeId;
6     std::string nodeType;
7     std::string oldValue;
8     std::string newValue;
9     int parentId; // target parent for INSERT/MOVE
10    int pos;      // target child index
11 };

```

Listing 3. ASTDiffOp from diff_core.h.

D. GumTree Matching Context

The GumTree implementation maintains bidirectional node mappings in a GumTreeContext struct:

```

1 struct GumTreeContext {
2     std::unordered_map<int, int> A_to_B;
3     std::unordered_map<int, int> B_to_A;
4     std::unordered_map<int, std::shared_ptr<ASTNode>>
5     nodes_A;
6     std::unordered_map<int, std::shared_ptr<ASTNode>>
7     nodes_B;
8     std::unordered_map<int, std::set<int>>
9     descendants_A;
10    std::unordered_map<int, std::set<int>>
11    descendants_B;
12 };

```

Listing 4. GumTreeContext from gumtree.cpp.

V. MYERS DIFF ALGORITHM

A. Overview

Myers’ algorithm [1] computes a shortest edit script (SES) between two sequences A (length N) and B (length M) by exploring the edit graph in a diagonal-layered BFS. Each diagonal $k = x - y$ represents a possible combination of x deletions from A and y insertions from B . The algorithm extends each diagonal as far as possible along matching (“snake”) steps before recording the frontier, then backtracks through the stored history to reconstruct the path.

Algorithm 1: Myers Diff — Forward Pass

Input: Sequence $A[0..N - 1]$, sequence $B[0..M - 1]$
Output: Ordered list of (x, y) waypoints on the shortest edit path

```

MAX ← N + M;
V ← 02·MAX+1; // diagonal frontier,
               // offset by MAX
history ← [];
for d ← 0 to MAX do
    history.push(V);
    for k ← -d to d step 2 do
        if k = -d or (k ≠ d and V[k - 1] < V[k + 1])
            then x ← V[k + 1]; // come from
            diagonal k + 1 (insertion)
            ;
        else x ← V[k - 1] + 1; // come from
            diagonal k - 1 (deletion)
            ;
        y ← x - k;
        while x < N and y < M and A[x] = B[y] do
            x += 1;
            y += 1; // consume snake
        end
        V[k] ← x;
        if x ≥ N and y ≥ M then return backtrack
            from history;
    end
end

```

B. Backtracking

After the forward pass terminates at depth D , the algorithm reconstructs the path by iterating backwards through the stored V snapshots. At each depth level d , it re-derives whether the current diagonal was reached by a downward (insertion) or rightward (deletion) move, then follows diagonal (snake) steps within that level. The resulting waypoints are reversed to yield forward order.

C. Output Encoding

The waypoint sequence is converted to a LineDiff vector: a diagonal step $\Delta x = 1, \Delta y = 1$ becomes EQUAL; a horizontal step $\Delta x = 1, \Delta y = 0$ becomes DELETE; a vertical step $\Delta x = 0, \Delta y = 1$ becomes INSERT.

D. Complexity

- **Time:** $O(ND)$ in the forward pass, $O(D^2)$ for the history storage and backtracking scan.
- **Space:** $O(N + M + D^2)$ due to the stored V snapshots.

In the worst case (no common lines), $D = N + M$, giving $O((N + M)^2)$ time and space. For typical code-review inputs where $D \ll N + M$, the algorithm is significantly faster.

VI. HISTOGRAM DIFF ALGORITHM

A. Overview

The Histogram algorithm (histogram.cpp) is a divide-and-conquer strategy inspired by the patience sorting approach

popularised in Git. It selects a *pivot*: the line that appears the fewest times in the current region of A , among lines that also appear in the corresponding region of B . The two inputs are then split at the pivot line, and the algorithm recurses on each sub-region independently. When no common line exists in a sub-region, the algorithm falls back to Myers diff on that sub-region.

Algorithm 2: Histogram Diff — Recursive Core

Input: $A[sa..ea)$, $B[sb..eb)$, result list
 Consume common prefix and suffix;
if $sa = ea$ **then** append INSERTs for remaining B ;
 return;
if $sb = eb$ **then** append DELETEs for remaining A ;
 return;
 Build frequency tables $freq_A$, $freq_B$ for current windows;
 Find rarest shared line ℓ^*
 ($\min_l |\{i : A[i] = l, freq_B(l) > 0\}|$);
if no shared line then
 Fall back to `myers_diff( $A[sa..ea)$ ,  $B[sb..eb)$ )`;
else
 Find pivot pair (i^*, j^*) : occurrence of ℓ^* minimising
 $|(i - sa)/(ea - sa) - (j - sb)/(eb - sb)|$;
 Recurse on $A[sa..i^*)$, $B[sb..j^*)$;
 Emit EQUAL for ℓ^* ;
 Recurse on $A[i^* + 1..ea)$, $B[j^* + 1..eb)$;
end
 Append reversed suffix equalities;

B. Pivot Selection Heuristic

Among all candidate occurrences of the rarest line in A and B , the pivot pair is chosen to minimise the difference in *relative position* within their respective windows:

$$(i^*, j^*) = \arg \min_{i,j} \left| \frac{i - sa}{ea - sa} - \frac{j - sb}{eb - sb} \right|.$$

This heuristic tends to produce more balanced recursive splits and better visual alignment of changed regions.

C. Complexity

Let $n = N + M$. In the best case (many unique shared lines at balanced positions), the recursion depth is $O(\log n)$ with $O(n)$ work at each level, giving $O(n \log n)$ total. In the worst case (no shared lines, full Myers fallback on every sub-region), the complexity degrades to $O(n^2)$. Prefix/suffix stripping reduces constant factors substantially on typical inputs.

VII. AST DESIGN AND PARSING

A. Dual-Mode Parser Dispatch

The entry point `build_ast()` in `ast.cpp` dispatches to one of two parsers based on the first non-whitespace character of the input:

- **S-expression parser** (`parse_sexpr`): invoked when the input begins with `(`. Processes Lisp-style nested expressions of the form `(TYPE [VALUE] child1 ... childk)`.

- **Toy C-like parser** (`parse_toy_code`): invoked otherwise. Handles a subset of C-like syntax including function, var/let, if/else, while, return, assignment, and binary expressions.

If the chosen parser returns an empty tree, the input falls back to `parse_fallback_text`, which tokenises the input line-by-line and word-by-word into a shallow Program $\rightarrow$ Line $\rightarrow$ Word hierarchy.

B. S-Expression Parser

`tokenize_sexpr` recognises four token types: open parenthesis, close parenthesis, quoted strings, and bare identifiers/numbers. `parse_sexpr_tokens` performs a top-down recursive descent: each non-parenthesised token becomes a `Literal` leaf; each `(TYPE [VALUE] children...)` group becomes an internal node with the given type, optional value, and recursively parsed children.

C. Toy C-like Parser

`tokenize_toy` classifies characters into six token types: KEYWORD, IDENTIFIER, NUMBER, STRING, OPERATOR, and PUNCTUATION. The keyword set is {function, var, let, if, else, while, return}. Single-line comments (`//`) are skipped during tokenisation.

`ToyParser` is a hand-written recursive-descent parser that builds the following node types:

TABLE I
AST NODE TYPES PRODUCED BY THE TOY C-LIKE PARSER

Node Type	Triggered By
Program	Top-level root
FunctionDecl	function name(...) {...}
Parameters	Formal parameter list
Block	Braced statement sequence
VarDecl	var name or let name
IfStatement	if (cond) then [else]
WhileStatement	while (cond) body
ReturnStatement	return [expr]
Assign	name = expr
BinaryExpr	expr op expr
Identifier	Bare name token
Literal	Number or string literal

Binary expressions are parsed with precedence climbing: comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) have precedence 1; additive operators (`+`, `-`) have precedence 2; multiplicative operators (`*`, `/`) have precedence 3.

D. Post-Processing

After parsing, `post_process()` is called recursively on the root. It sets each node's parent weak pointer, accumulates subtree size (number of nodes), computes height (longest root-to-leaf path), and builds `structure_hash`:

$$h(v) = \text{type} : \text{value}[h(c_1), h(c_2), \dots, h(c_k)].$$

Two subtrees are isomorphic if and only if their hashes are equal.

a) *Complexity of post-processing*: $O(n)$ time and space, where n is the number of AST nodes.

VIII. GUMTREE AST DIFF ALGORITHM

A. Overview

The GumTree diff proceeds in three phases: (1) top-down subtree matching, (2) bottom-up node matching, and (3) edit-script generation. All state is maintained in a `GumTreeContext` (Section IV). Before either matching phase, `cache_descendants` recursively populates the descendant sets and node maps.

B. Phase 1: Top-Down Matching

Algorithm 3: GumTree Top-Down Phase

Input: Trees T_1, T_2 , minimum height $h_{\min} = 2$
 Collect all nodes of T_1 into N_1 , all nodes of T_2 into N_2 ;
 $h_{\max} \leftarrow \max_{n \in N_1 \cup N_2} n.height$;
for $h \leftarrow h_{\max}$ **to** $h_{\min}$ **do**
 $H_1 \leftarrow \{n \in N_1 : n.height = h, n \notin matched_A\}$;
 $H_2 \leftarrow \{n \in N_2 : n.height = h, n \notin matched_B\}$;
 foreach $n_1 \in H_1$ **do**
 foreach $n_2 \in H_2$ **do**
 if $n_1.structure_hash = n_2.structure_hash$
 then Map entire subtree of n_1 onto
 subtree of n_2 ; **break** ;
 end
 end
 end
end

The subtree mapping is applied recursively: if the root hashes match, children are mapped by index up to the minimum child count of the two nodes.

C. Phase 2: Bottom-Up Matching

Unmatched nodes in T_1 are visited in post-order. For each unmatched node n_1 , all unmatched nodes $n_2 \in T_2$ of the same type are evaluated using the Dice coefficient over the matched descendants:

$$\text{sim}(n_1, n_2) = \frac{2 \cdot |\{d \in \text{desc}(n_1) : \text{ctx.A_to_B}[d] \in \text{desc}(n_2)\}|}{|\text{desc}(n_1)| + |\text{desc}(n_2)|}.$$

A match is established when $\text{sim}(n_1, n_2) \geq 0.5$. After parent-level matching, a recovery pass attempts to match unmatched children of the newly matched pair by comparing type and value directly. A final leaf-recovery pass matches remaining unmatched leaves with identical type and value.

D. Phase 3: Edit Script Generation

Deletions are emitted for every unmatched node in T_1 (post-order). The algorithm then traverses T_2 in pre-order:

- A *matched* node is checked for a value change (UPDATE if $n_1.value \neq n_2.value$) and a structural displacement (MOVE if the matched counterpart in T_1 has a different parent or position).
- An *unmatched* node receives an INSERT operation with its parent's mapped ID and its index within the parent's children.

E. Complexity

Let n_A and n_B denote the node counts of T_1 and T_2 .

- **Descendant caching**: $O(n_A + n_B)$.
- **Top-down phase**: $O(n_A \cdot n_B)$ in the worst case (all nodes at the same height); $O((n_A + n_B) \log(n_A + n_B))$ on balanced trees.
- **Bottom-up phase**: $O(n_A \cdot n_B)$ comparisons for similarity, $O(n_A + n_B)$ for the child-recovery and leaf-recovery passes.
- **Edit script generation**: $O(n_A + n_B)$.
- **Overall**: $O(n_A \cdot n_B)$.

IX. BACKEND IMPLEMENTATION

A. REST API

The Express.js server (`server.js`) exposes four endpoints (Table II).

TABLE II
REST API ENDPOINTS

Method	Route	Description
POST	/api/diff	Submit a diff request
GET	/api/history	List past comparisons (metadata)
GET	/api/history/:id	Retrieve full comparison record
DELETE	/api/history/:id	Delete a comparison record

B. Diff Request Lifecycle

Figure 2 summarises the full request lifecycle for a line-based diff.

1. Client sends POST /api/diff with {title, textA, textB, algorithm}
2. Server validates fields and selects handler:
 - (a) complexity: run JS estimator, skip engine
 - (b) myers|histogram|gumtree: continue
3. Write textA $\rightarrow$ temp/temp_{ts}_A.txt
Write textB $\rightarrow$ temp/temp_{ts}_B.txt
4. `execFile(enginePath, [algo, fileA, fileB])`
5. Parse JSON from stdout
6. Run JS complexity estimator on both texts
7. Delete temporary files
8. Persist record to MongoDB (or in-memory fallback)
9. Return full record as JSON response

Fig. 2. Diff request lifecycle in `server.js`.

The `execFile` call uses a buffer limit of 10 MB to accommodate large AST outputs. The temporary-file timestamps are millisecond-precision Unix timestamps, providing effectively unique names for concurrent requests.

C. MongoDB Schema and Fallback

The Mongoose schema `Comparison` stores: `title` (String), `textA` (String), `textB` (String), `algorithm` (enum: `myers`, `histogram`, `gumtree`), `diffResult` (Mixed), and `createdAt` (Date).

If the initial `mongoose.connect()` call rejects, the server sets `dbConnected = false` and uses a module-level `memoryHistory` array for all subsequent history operations. The `GET /api/history` route omits `textA`, `textB`, and `diffResult` from the listing response to keep list payloads small, consistent between the MongoDB and in-memory paths.

```

1 const ComparisonSchema = new mongoose.Schema({
2   title: { type: String, required: true },
3   textA: { type: String, required: true },
4   textB: { type: String, required: true },
5   algorithm: {
6     type: String,
7     enum: ['myers', 'histogram', 'gumtree'],
8     required: true
9   },
10  diffResult: {
11    type: mongoose.Schema.Types.Mixed,
12    required: true
13  },
14  createdAt: { type: Date, default: Date.now }
15 });

```

Listing 5. Mongoose schema from `models/Comparison.js`.

Note that the schema enum does not include `complexity`: complexity requests are handled entirely in JavaScript and returned without database persistence beyond the in-memory list.

X. FRONTEND IMPLEMENTATION

A. Application State

`App.jsx` maintains the following React state variables: `title`, `algorithm`, `textA`, `textB`, `history`, `loading`, `diffResult`, `activeHistoryId`, `error`, `fileName`, `fileBName`. File upload is handled via `FileReader.readAsText` with a 1 MB size guard. Accepted file extensions are `.txt`, `.cpp`, `.c`, `.java`, `.py`, `.js`, `.ts`.

Three built-in sample inputs are available to the user (Table III).

TABLE III
BUILT-IN SAMPLE INPUTS IN `APP.JSX`

Key	Algorithm	Purpose
text	histogram	Prose line-diff demonstration
code	gumtree	C-like refactoring (variable rename, assignment addition, return change)
sexpr	gumtree	S-expression tree comparison

B. DiffViewer Component

`DiffViewer.jsx` renders the output of Myers or Histogram diff in one of two modes toggled by the user:

a) *Unified View*: A single table with two line-number columns (File A and File B) and a content column. Each row is coloured green (INSERT), red (DELETE), or neutral (EQUAL), with a `+/-/space` prefix sign.

b) *Split View*: Two vertically synchronised panes. The left pane shows File A content, highlighting DELETE rows; the right pane shows File B content, highlighting INSERT rows. An alignment step groups consecutive non-equal operations into *chunks* and pads the shorter side of each chunk with empty rows, so that corresponding changes in *A* and *B* appear at the same vertical position.

c) *Statistics Bar*: A summary bar counts the number of inserted, deleted, and unchanged lines using a single `reduce` pass over the diff array.

C. ASTDiffViewer Component

`ASTDiffViewer.jsx` renders the GumTree output as two parallel interactive trees. On mount, a `useEffect` hook reconstructs approximate node mappings in the frontend by intersecting the pre-order traversal of T_A (filtered to exclude deleted nodes) with the pre-order traversal of T_B (filtered to exclude inserted nodes) and pairing them positionally. This frontend reconstruction is approximate and is used only for hover-highlight linking; the authoritative edit script is the one produced by the C++ engine.

Each node is rendered as a clickable item with a collapse/expand toggle. The node background is colour-coded by operation type: red for DELETE, green for INSERT, amber for UPDATE, blue for MOVE. Hovering a node in one tree highlights the linked node in the other tree. An *Edit Script Operations* panel below the trees lists every operation produced by the C++ engine with its type, node ID, node type, old value, new value, target parent ID, and target position.

D. ComplexityViewer Component

`ComplexityViewer.jsx` displays the results of the JavaScript heuristic estimator (Section XI). Each file panel shows the estimated complexity class as a coloured pill, a confidence percentage, and the list of detection reasons. A verdict panel at the top compares the two estimated complexities using a fixed ordinal ranking table and reports which file is estimated to have the lower asymptotic class, or announces equivalence.

XI. SUPPLEMENTARY ANALYSIS FEATURES: HEURISTIC COMPLEXITY ESTIMATION

The `estimateComplexity` function in `server.js` performs lightweight heuristic static analysis of a submitted code string and returns an estimated asymptotic time complexity class, a numeric confidence score (0–100), and a list of human-readable detection reasons.

This module is *not* part of the core diff engine. It is an educational utility that assists users in comparing the estimated asymptotic efficiency of two code snippets submitted for diff comparison.

A. Detection Rules

The estimator applies pattern matching in priority order (Table IV). Rules are evaluated top-to-bottom; the first matching rule determines the output.

TABLE IV
HEURISTIC COMPLEXITY DETECTION RULES (IN PRIORITY ORDER)

Detected Pattern	Estimate	Conf.
priority_queue + adj + dist keywords	$O(E \log V)$	80%
dfs(, bfs(, or queue< keywords	$O(V + E)$	85%
parent + find(+ union-variant keywords	$O(\alpha(n))$	85%
segment or tree[4*n] or query(+update())	$O(\log n)$	85%
Function name with ≥ 3 calls (branching recursion)	$O(2^n)$	75%
Function name with exactly 2 calls (linear recursion)	$O(n)$	75%
$i*i \leq$ or $j*j \leq$ pattern + nested loop	$O(n\sqrt{n})$	80%
$i*i \leq$ or $j*j \leq$ pattern alone	$O(\sqrt{n})$	85%
$i* = / j* =$ or $/ = 2$ + nested loop	$O(n \log n)$	80%
$i* = / j* =$ or $/ = 2$ alone	$O(\log n)$	85%
sort(or std::sort(+ outer loop	$O(n^2 \log n)$	75%
sort(or std::sort(alone	$O(n \log n)$	95%
binary_search(, lower_bound(, upper_bound(	$O(\log n)$	90%
Maximum loop nesting depth ≥ 3	$O(n^3)$	90%
Maximum loop nesting depth = 2	$O(n^2)$	95%
Maximum loop nesting depth = 1	$O(n)$	95%
No loops detected	$O(1)$	80%

Loop nesting depth is tracked by counting `for\(\` and `while\(\` occurrences and decrementing a depth counter on each `\)` character encountered in the source lines.

B. Limitations

The estimator is a heuristic. It does not parse the code and may produce incorrect results for programs whose complexity does not match any of the listed patterns, programs using container methods or recursion in atypical forms, and code written in languages other than C or C++. The confidence values are hand-assigned constants, not derived from empirical evaluation. The module is provided as an educational aid and should not be used as a substitute for formal complexity analysis.

XII. JSON IPC PROTOCOL

Communication between the Node.js backend and the C++ engine is performed via `stdout` JSON. The engine writes one JSON object per invocation.

A. Line-Diff Response (Myers / Histogram)

```

1 {
2   "algorithm": "myers",
3   "diff": [
4     {
5       "type":      "equal" | "delete" | "insert",
6       "line":      "<escaped line content>",
7       "lineNoA":   <int>,    // 1-based; 0 if not
                        present in A
8       "lineNoB":   <int>     // 1-based; 0 if not
                        present in B
9       }, ...
10  ]
11 }
```

Listing 6. JSON schema for a line-based diff response.

B. AST Diff Response (GumTree)

```

1 {
2   "algorithm": "gumtree",
3   "treeA": { <ASTNode JSON> },
4   "treeB": { <ASTNode JSON> },
5   "diff": [
6     {
7       "type":      "insert" | "delete" | "update" | "move",
8       "nodeId":    <int>,
9       "nodeType":  "<string>",
10      "oldValue":   "<string>",
11      "newValue":   "<string>",
12      "parentId":   <int>,
13      "pos":        <int>
14    }, ...
15  ]
16 }
```

Listing 7. JSON schema for a GumTree diff response.

An `ASTNode` in JSON has the form `{"id":<int>,"type": "<str>","value": "<str>","children": [...] }`. String fields are escaped using an inline `escapeJSON` function defined in `diff_core.h`, which handles ```, control characters, and Unicode code points below U+0020.

XIII. EXPERIMENTAL EVALUATION

This section characterises the system’s behaviour based on the implementation. No external benchmark datasets or fabricated execution times are used.

A. Algorithm Selection Guide

The three line-based and structural modes serve distinct use cases, as summarised in Table V.

TABLE V
ALGORITHM SELECTION GUIDE

Algorithm	Appropriate Input and Use Case
Myers	Text or code where minimal edit count matters; output is compact but may split logical changes across multiple edit groups.
Histogram	Code reviews where visual grouping of changes is more important than minimality; anchors on rare lines to keep related changes contiguous.
GumTree	Structural refactorings: renames, moves, insertions and deletions of entire subtrees; meaningful only when input is valid toy C-like code or an S-expression.

B. AST Parsing Coverage

The dual-mode parser accepts:

- **S-expressions:** any parenthesised nested structure, e.g. `(Program (FunctionDecl main (Parameters ...)))`.

- **Toy C-like code:** programs using `function`, `var/let`, `if/else`, `while`, `return`, assignments, and binary expressions with the operators listed in Section VII.
- **Fallback:** arbitrary text, decomposed into a `Program` $\rightarrow$ `Line` $\rightarrow$ `Word` tree.

The three built-in samples (Table III) exercise all three input paths.

C. GumTree Edit Script Characteristics

The GumTree engine produces four operation types. Table VI summarises when each is emitted.

TABLE VI
GUMTREE EDIT SCRIPT OPERATION TYPES

Operation	Condition
DELETE	Node in T_1 has no match in T_2 after both phases
INSERT	Node in T_2 has no match in T_1 after both phases
UPDATE	Node matched; $n_1.value \neq n_2.value$
MOVE	Node matched; parent or child-position differs from expectation

D. Complexity Estimation Coverage

The heuristic estimator covers eight distinct complexity classes ($O(1)$, $O(\log n)$, $O(\sqrt{n})$, $O(n)$, $O(V + E)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(E \log V)$, $O(\alpha(n))$, $O(n\sqrt{n})$, $O(n^2 \log n)$) through seventeen distinct pattern rules applied in the priority order shown in Table IV.

E. History Persistence Modes

Table VII compares the two history persistence modes.

TABLE VII
HISTORY PERSISTENCE MODE COMPARISON

Aspect	MongoDB Mode	In-Memory Mode
Persistence	Survives server restart	Lost on restart
Storage backend	MongoDB 7, Mongoose 8	JS Array
ID format	BSON ObjectId	mem_{timestamp}
List endpoint	DB query, sorted by date	Array slice
Single fetch	findById	Array find
Delete	findByIdAndDelete	Array splice
Activation	Successful mongoose.connect	Connection failure

XIV. LIMITATIONS

- 1) **Parser scope.** The toy C-like parser supports only the keywords and constructs listed in Section VII. Constructs such as `for` loops, `do...while`, `class` declarations, templates, and preprocessor directives are not recognised and fall back to the word-tree parser, which may produce less meaningful structural diffs.
- 2) **Windows-only binary.** The pre-compiled executable is named `diff_engine.exe`, targeting Windows; Unix-based deployments must recompile from source using the provided `g++` command.

- 3) **No word-level diff.** Line-based modes operate on complete lines; intra-line changes are not identified.
- 4) **GumTree matching quality.** The implementation uses a positional index mapping for tie-breaking in the top-down phase rather than the candidate-list approach described in the original GumTree paper, which may produce suboptimal matches on deeply nested trees with many siblings of the same type.
- 5) **Complexity estimator.** The heuristic estimator performs no semantic analysis and is not sound. It may misclassify or fail to classify programs that do not match its pattern set.
- 6) **File size limit.** The frontend enforces a 1 MB per-file upload limit. The `execFile` call supports up to 10 MB of JSON output.
- 7) **Concurrency.** Temporary files are named by millisecond timestamp, which is not guaranteed unique under high concurrency.

XV. CONCLUSION

This paper has described the complete implementation of DiffEngine Dashboard, an open-source platform integrating three differencing algorithms—Myers, Histogram, and GumTree AST Diff—within a MERN-stack web application. The core algorithms are implemented in C++14, compiled to a standalone executable, and invoked by the Node.js backend over a JSON IPC protocol. A dual-mode AST parser handles both a toy C-like language and Lisp-style S-expressions, enabling structural comparison through the GumTree two-phase matching approach. The React frontend renders line-based diffs in split and unified views and displays annotated, interactive AST trees. Comparison history is persisted in MongoDB with transparent in-memory fallback. A supplementary heuristic complexity estimator provides educational side-by-side asymptotic class comparison, implemented entirely in JavaScript and clearly separated from the core diff engine.

The system is intended as an educational and research tool demonstrating the integration of compiled algorithms with a modern web stack. Directions for future work include word-level differencing within changed lines, integration of a full language parser (e.g. via tree-sitter) to extend GumTree coverage to real-world programming languages, and a more rigorous candidate-list approach for the GumTree top-down matching phase.

REFERENCES

- [1] E. W. Myers, “An $O(ND)$ difference algorithm and its variations,” *Algorithmica*, vol. 1, no. 1–4, pp. 251–266, 1986.
- [2] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, “Fine-grained and accurate source code differencing,” in *Proc. 29th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, Vasteras, Sweden, Sep. 2014, pp. 313–324.
- [3] JGit Contributors, “JGit Histogram diff implementation,” Eclipse Foundation, 2010. [Online]. Available: <https://git.eclipse.org/r/jgit/jgit>
- [4] S. S. Chawathe, A. Rajaraman, H. Garcia-Molina, and J. Widom, “Change detection in hierarchically structured information,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Montreal, Canada, Jun. 1996, pp. 493–504.

- [5] H. Gall, K. Hajek, and M. Jazayeri, "Detection of logical coupling based on product release history," in *Proc. IEEE Int. Conf. Software Maintenance (ICSM)*, Oxford, UK, Aug. 1999, pp. 190–198.
- [6] M. Hashimoto and A. Mori, "Diff/TS: A tool for fine-grained structural change analysis," in *Proc. 15th Working Conf. Reverse Engineering (WCRE)*, Antwerp, Belgium, Oct. 2008, pp. 279–288.
- [7] M. Martinez, L. Duchien, and M. Monperrus, "Automatically extracting instances of code change patterns with AST analysis," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, Victoria, Canada, Sep. 2014, pp. 388–391.